

```
from sklearn.ensemble import IsolationForest
# Detect anomalies
isolation_forest = IsolationForest(random_state=42)
isolation_forest.fit(X)
anomalies = isolation_forest.predict(X)
print(anomalies)
```

Natural language processing (NLP): Although not specifically designed for NLP, Scikit-learn can preprocess text data, perform feature extraction (using CountVectorizer or TfidfVectorizer), and train classifiers for tasks like sentiment analysis, spam detection, or document categorisation.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Extract features from text data
texts = ["This is great", "I hate this", "Amazing experience"]
vectorizer = TfidfVectorizer()
X_text = vectorizer.fit_transform(texts)
print(X_text.toarray())
```

Time series analysis: While Scikit-learn does not have native support for time series forecasting, it can preprocess and transform time-series data to be used with machine learning models. Tasks like sales forecasting or energy usage prediction can be tackled by converting time-series data into supervised learning problems.

Healthcare applications: Diagnostic models can be built to classify medical conditions, predict patient outcomes, or analyse genetic data. For example, Scikit-learn is often used in predictive modelling for patient readmissions or disease progression analysis.

Image recognition and computer vision: Scikit-learn can be used in combination with feature extraction tools like SIFT or ORB to classify images or detect patterns in visual data. It is often used for tasks like defect detection in manufacturing or classifying satellite imagery.

How Scikit-learn outperforms other libraries

Scikit-learn excels in many aspects compared to other libraries, making it a preferred choice for traditional machine learning tasks. Here's how it stands out.

Ease of use: Scikit-learn's unified API design ensures that algorithms and functions work in a consistent manner, reducing the learning curve for new users. For example, both a linear regression model (LinearRegression) and a decision tree (DecisionTreeClassifier) use the same `.fit()` and `.predict()` methods, simplifying the workflow.

Comprehensive feature set: While libraries like TensorFlow or PyTorch focus on deep learning, Scikit-learn provides tools for preprocessing, feature selection, clustering,

classification, and regression under one roof. For example, it combines preprocessing functions like StandardScaler and model tuning tools like GridSearchCV.

Lightweight and fast for traditional ML: Unlike deep learning libraries that are computationally intensive, Scikit-learn is optimised for traditional ML algorithms and works efficiently for medium-sized datasets. As an example, training a Random Forest classifier on the Iris dataset is quick and requires minimal setup.

Strong documentation and community support: Scikit-learn boasts extensive, beginner-friendly documentation with numerous examples, making it accessible for users of all skill levels. Its widespread adoption ensures access to community-driven tutorials, forums, and third-party guides.

Seamless integration: Scikit-learn integrates smoothly with Pandas for data manipulation, NumPy for numerical computations, and Matplotlib for visualisation. As an example, you can pass a Pandas DataFrame directly to Scikit-learn functions without additional conversion.

Broad algorithm support: Scikit-learn supports both basic algorithms (like linear regression and K-means) and advanced ones (like Gradient Boosting and Support Vector Machines). This versatility often reduces the need for additional libraries. An example is performing classification, clustering, and dimensionality reduction without switching tools.

Customisability and extensibility: It allows users to define custom transformers and estimators, which can be integrated into the library's pipelines. Creating a custom feature transformation using TransformerMixin is an example of this.

Comparison with specialised libraries like TensorFlow/PyTorch and XGBoost/LightGBM: While Scikit-learn lacks the deep learning capabilities of TensorFlow and PyTorch, it is significantly easier to use for traditional ML tasks and requires less computational power. XGBoost/LightGBM libraries specialise in gradient boosting, but Scikit-learn supports similar ensemble methods, making it versatile for diverse ML tasks.

Limitations of Scikit-learn

Not ideal for Big Data: Scikit-learn works best with datasets that fit into memory. For larger datasets, distributed frameworks like PySpark or Dask-ML are more suitable.

Limited deep learning support: While excellent for traditional machine learning, it doesn't support deep learning. Libraries like TensorFlow and PyTorch are better suited for neural networks and deep learning applications.

No GPU acceleration: Scikit-learn's operations are CPU-bound, making it slower for very large datasets or complex computations compared to GPU-accelerated libraries. 

 By: Rajnesh Devi

The author is an assistant professor at the Yamuna Group of Institutions.